PBL CASE STUDY | SOFTWARE ENGINEERING

Academic Year 2026-27

Online Food Ordering System

A Software Engineering Case Study

Requirements, modelling, design, testing and maintenance.

Submitted to

Mr. Jitendra

Submitted by

Garima

UID 25BCD10246

Section 25BCD-4A

Department of BCA (Data Science)

Chandigarh University

Acknowledgement

I would like to thank my subject teacher, Mr. Jitendra, for his guidance and for giving us the chance to work on a Project Based Learning case study. His feedback helped me understand how the stages of software engineering fit together in a real project. I also thank my department, my classmates and my family for their support while I prepared this report.

Abstract

Ordering food online has become part of everyday life. Customers want to browse menus, place an order, pay and follow the delivery from their phone, while restaurants want a steady flow of orders and a simple way to manage them. This case study applies software engineering methods to the design of an Online Food Ordering System that serves customers, restaurants, delivery partners and an administrator.

The report follows the stages of the software development life cycle. It begins with the problem and a feasibility study, selects the Agile (Scrum) process model, and prepares a requirement analysis with functional and non-functional requirements. The system is then modelled with a use case diagram, a data flow diagram and an entity-relationship diagram, and designed as a three-tier architecture with a relational database. The report also gives a test plan, a risk analysis, a project schedule and notes on maintenance, and ends with the lessons learned and the scope for future work.

Table of Contents

Acknowledgement 2

Abstract 2

Table of Contents 3

1. Introduction 3

2. Study of Existing Systems 4

3. Feasibility Study 5

4. Software Process Model 6

5. Requirement Analysis 7

6. System Modelling 9

7. System Design 12

8. Technology Stack and Implementation Plan 15

9. Testing 16

10. Risk Analysis and Project Schedule 17

11. Maintenance and Security 17

12. Learning Outcomes and Future Enhancements 19

13. Conclusion 19

14. References 19

1. Introduction

1.1 Background

Before online ordering, a customer who wanted a meal from a restaurant had to visit it or phone it. Phone orders depend on a person writing down the order correctly, menus are often out of date, and there is no easy way to check the status of an order. Smartphones, mobile internet and digital payments have changed this. Today, a customer can compare restaurants, read the menu with prices, pay online and watch the order move from the kitchen to the doorstep.

For a student of software engineering, an online food ordering system is a useful case study. It has several kinds of users with different needs, it handles money and personal data, and it must work reliably at busy hours. These are the same concerns that real software projects face.

1.2 About This Case Study

This report is prepared as a Project Based Learning (PBL) case study for the subject of software engineering. The aim is not only to describe a food ordering website but also to show how a team would plan, analyse, design, test and maintain it, using the standard tools of the subject.

1.3 Problem Statement

Small and medium restaurants often take orders by phone or on paper. This leads to wrong orders, long waiting times, lost orders at busy hours and no record of sales. Customers cannot see the full menu, compare prices, or know when the food will arrive. Delivery is also hard to organise without a clear view of which orders are ready and which rider is free.

A single online system is needed that lets customers order and pay easily, lets restaurants manage menus and orders, assigns deliveries, and gives the administrator control and reports.

1.4 Objectives

1. To study the needs of customers, restaurants, delivery partners and administrators.
2. To prepare a clear requirement specification for an online food ordering system.
3. To model the system using use case, data flow and entity-relationship diagrams.
4. To design a three-tier architecture and a normalised database.
5. To plan testing, risk handling, scheduling and maintenance for the project.

1.5 Scope

The system covers registration and login, restaurant and menu browsing, a shopping cart, order placement, online and cash payment, order tracking, ratings, restaurant order management, delivery assignment and administrator reports. The system does not cover restaurant inventory, staff payroll, table booking or a loyalty programme. Some of these are listed as future enhancements in Chapter 12.

2. Study of Existing Systems

Customers today order food in three ways: by visiting or phoning the restaurant, through a restaurant’s own website, or through a large food delivery platform that lists many restaurants. Each way has strengths and weaknesses, which help to decide what the new system must provide.

Feature	Phone or walk-in	Restaurant’s own site	Large delivery platform
Menu with current prices	Often outdated	Available	Available
Choice of restaurants	One	One	Many
Online payment	No	Sometimes	Yes
Live order status	No	Rarely	Yes
Order history	No	Limited	Yes
Cost to the restaurant	Low	Setup cost	Commission on orders

The comparison shows that small restaurants either lack features or must pay commission. The proposed system gives them the main features of a large platform in a form that is easy to run and adapt.

3. Feasibility Study

3.1 Technical Feasibility

The system uses common web technologies: HTML, CSS and JavaScript for the interface, a server-side language for the application logic, and a relational database such as MySQL. Payment is handled through a payment gateway’s test mode during development. All of this can be learned and built by a student team, so the project is technically feasible.

3.2 Economic Feasibility

All development tools and frameworks are free. The running cost is limited to hosting, a domain name and gateway charges, which are paid from the orders. Restaurants save the cost of order takers and lose fewer orders, so the benefit is greater than the cost.

3.3 Operational Feasibility

The interface follows the pattern customers already know from other ordering apps, so little training is needed. Restaurant staff need only to learn the order screen and the menu editor. Because the system fits the way restaurants already work, it is operationally feasible.

3.4 Schedule Feasibility

The work can be divided into short sprints, with a working version available after the first few weeks. A twelve-week plan is given in Chapter 10.

3.5 Cost Estimation Using Basic COCOMO

The size of the system is estimated at about 8 thousand lines of code (8 KLOC). The system is small and the team is familiar with the technology, so the organic mode of the Basic COCOMO model is used, with the constants a = 2.4, b = 1.05, c = 2.5 and d = 0.38.

Quantity	Formula	Result
Effort (person-months)	E = a x (KLOC)^b = 2.4 x 8^1.05	about 21.3
Development time (months)	T = c x E^d = 2.5 x 21.3^0.38	about 8.0
Average team size	E / T	about 2.7 persons

The estimate shows that a team of three students would need about eight months of full-time work for the complete system. The twelve-week plan in Chapter 10 therefore delivers a first working release with the main features, and the remaining features follow in later sprints.

4. Software Process Model

4.1 Choice of Model

The Agile model with Scrum was chosen. Requirements for a food ordering system change as users give feedback: a restaurant may ask for a new payment option, or customers may want a filter for vegetarian dishes. A model that delivers small working parts and welcomes change suits this better than the Waterfall model, in which each stage is finished before the next begins.

Aspect	Waterfall	Agile (Scrum)
Requirements	Fixed at the start	Can change every sprint
Delivery	One release at the end	Working software after each sprint
Customer feedback	At the end	After every sprint
Risk	Found late	Found early
Fit for this project	Low	High

4.2 Scrum Roles and Sprints

The product owner decides the priority of features, the Scrum master keeps the process on track, and the development team builds and tests the work. The work is kept in a product backlog and delivered in sprints of two weeks.

Sprint	Goal
1	Registration, login, restaurant and menu listing
2	Cart, order placement and bill calculation
3	Payment, order tracking and notifications
4	Restaurant dashboard and delivery assignment
5	Administrator panel, ratings and reports
6	Testing, fixes, security review and release

5. Requirement Analysis

5.1 Stakeholders

Stakeholder	Need
Customer	Browse menus, order quickly, pay safely and track the order
Restaurant owner	Manage the menu, accept orders and view sales
Delivery partner	Receive delivery tasks and update their status
Administrator	Manage users, restaurants and reports

5.2 Functional Requirements

Customer module

- Register, log in and manage the profile and delivery addresses.
- Search restaurants and filter by cuisine, rating or price.
- View the menu, add items to the cart and change quantities.
- Place an order and pay online or in cash on delivery.
- Track the order status and view past orders.
- Rate the food and the delivery.

Restaurant module

- Add, edit, hide or delete menu items and prices.
- Accept or reject incoming orders.
- Update the status to preparing and ready for pickup.
- View daily orders and sales.

Delivery module

- Receive an assigned order with pickup and drop details.
- Mark the order as picked up and delivered.

Administrator module

- Approve or block restaurants, customers and delivery partners.
- View reports of orders, revenue and ratings.

5.3 Non-Functional Requirements

- **Performance:** pages should load within three seconds and the menu should respond immediately.
- **Security:** passwords are stored as hashes, the connection uses HTTPS, and card details are never stored by the system.
- **Reliability:** an order must never be lost or charged twice.
- **Usability:** a new customer should be able to place an order within a few minutes without help.
- **Scalability:** the system should handle many users at lunch and dinner time.
- **Portability:** the site must work on phones, tablets and computers.

5.4 Constraints and Assumptions

The system depends on the internet and on a third-party payment gateway. It is assumed that each restaurant has a device to receive orders and that each delivery partner has a smartphone.

6. System Modelling

6.1 Use Case Diagram

The use case diagram shows what each actor can do with the system. The customer places and tracks orders. The restaurant manages the menu and the orders. The delivery partner handles deliveries, and the administrator manages users and reports.

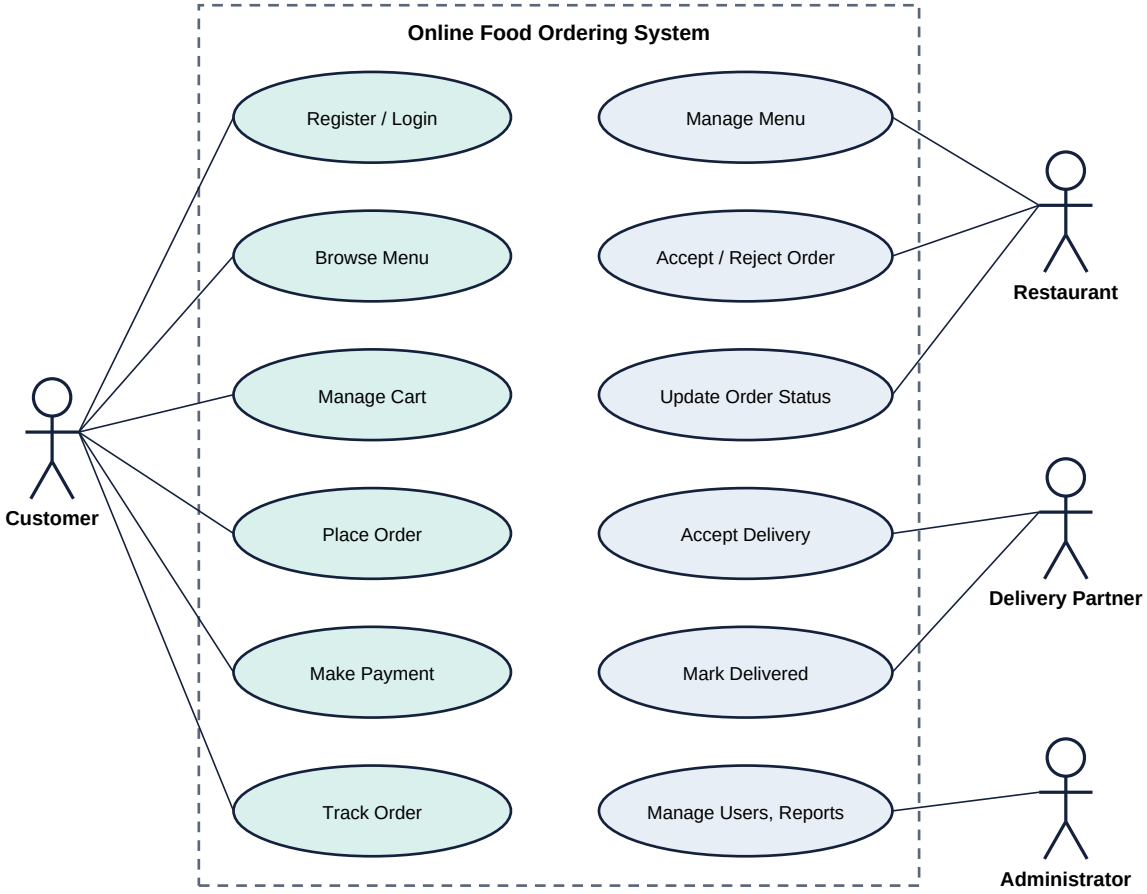

Figure 6.1: Use case diagram of the Online Food Ordering System

Use case	Description
Place order	The customer confirms the items in the cart, selects an address and a payment mode
Make payment	The system sends the amount to the payment gateway and records the result
Accept or reject order	The restaurant confirms that it can prepare the order
Update order status	The restaurant or the delivery partner moves the order to its next stage
Track order	The customer sees the current status and the expected time

6.2 Data Flow Diagram

The context diagram (level 0) shows the whole system as one process and the data that moves between it and the outside world.

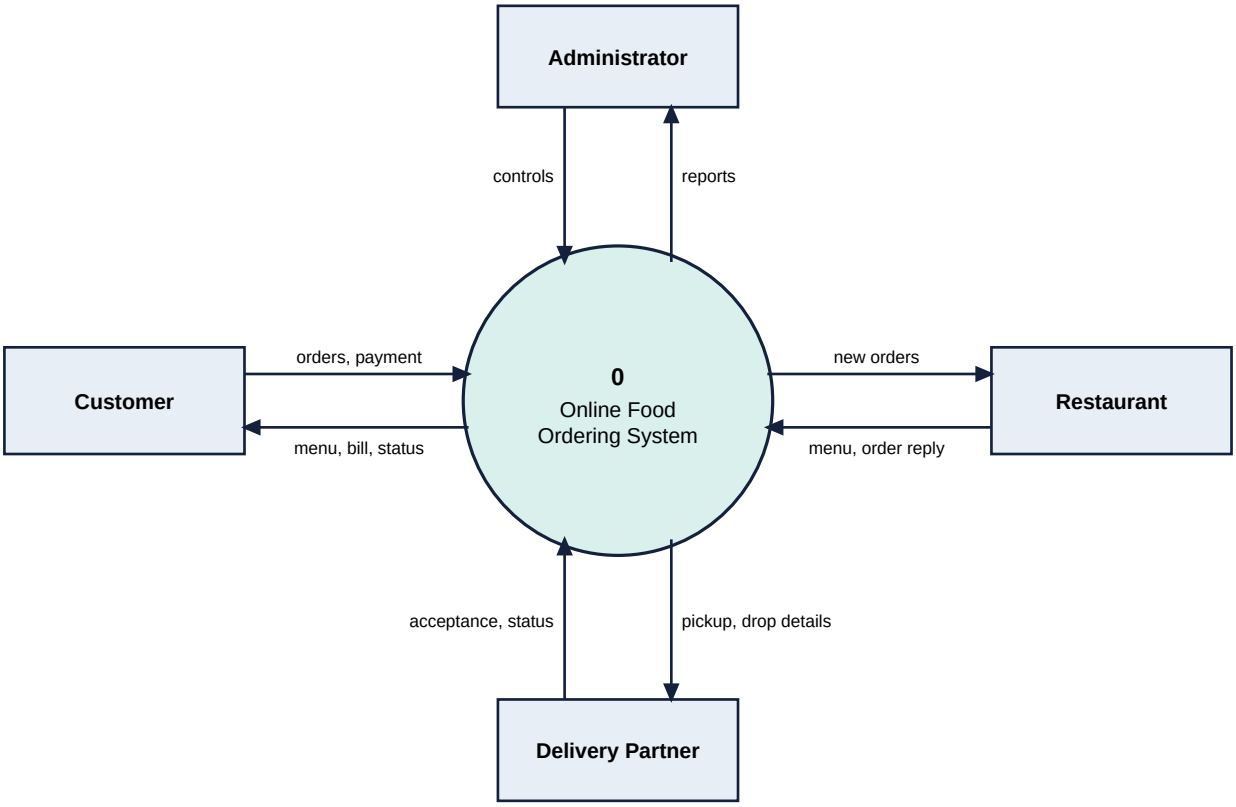

Figure 6.2: Data flow diagram, level 0

At level 1 the single process is divided into six processes.

Process	Input	Output
1.0 Manage accounts	Registration and login details	Verified user session
2.0 Manage menu	Menu changes from the restaurant	Updated menu
3.0 Process order	Cart, address and payment mode	Confirmed order and bill
4.0 Handle payment	Order amount	Payment result
5.0 Manage delivery	Ready orders, free partners	Delivery assignment and status
6.0 Generate reports	Order and payment records	Sales and rating reports

6.3 Entity-Relationship Diagram

The data of the system is held in seven entities. A customer places many orders. A restaurant receives many orders and offers many menu items. Each order contains many order items, has one payment and is delivered by one delivery partner.

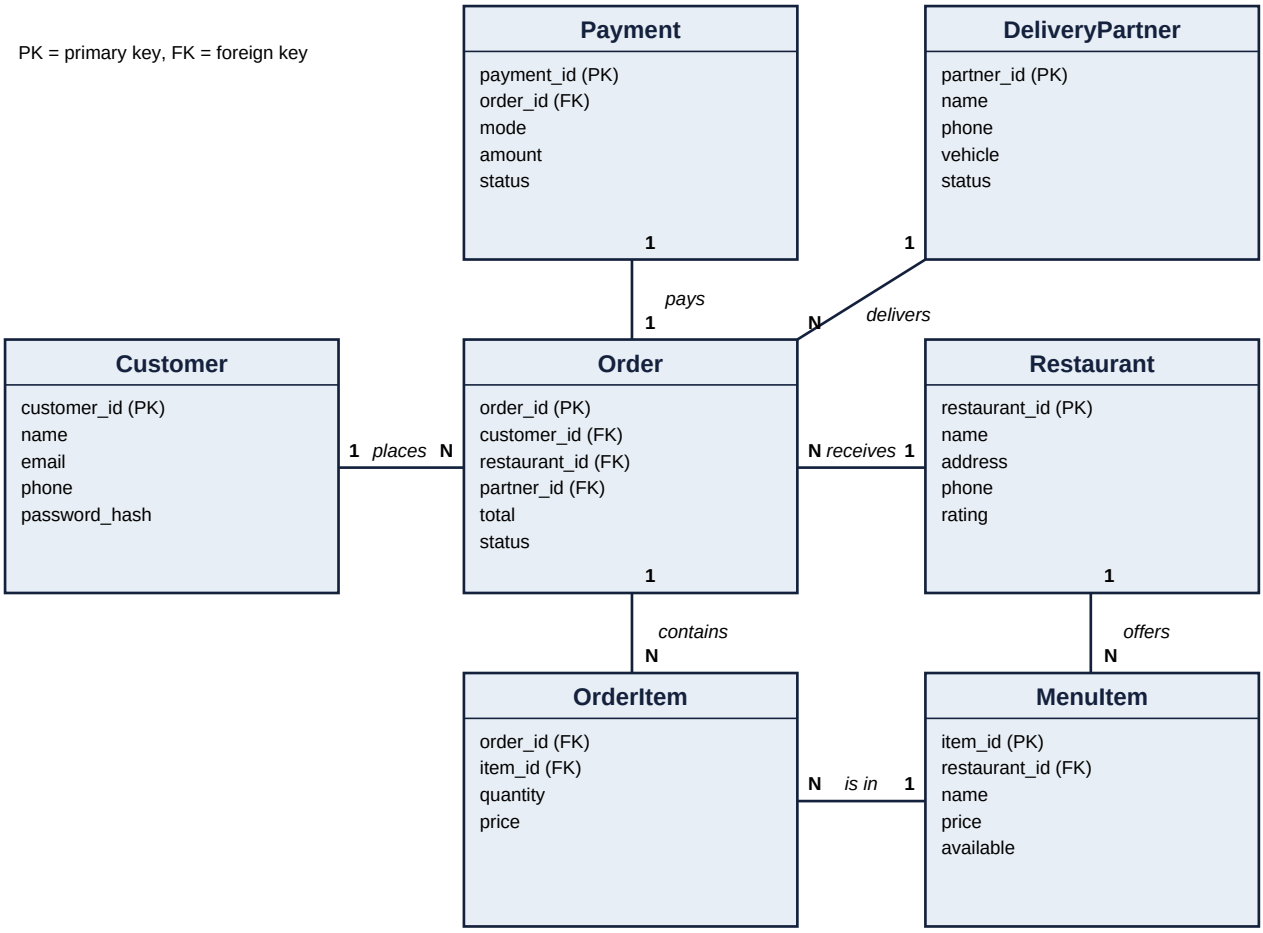

Figure 6.3: Entity-relationship diagram (PK primary key, FK foreign key)

7. System Design

7.1 Architecture

The system is divided into three tiers. The presentation tier contains the screens used by customers, restaurants, delivery partners and the administrator. The application tier holds the business rules in separate modules. The data tier stores everything in a relational database. External services, such as the payment gateway and the SMS or email service, are reached only from the application tier.

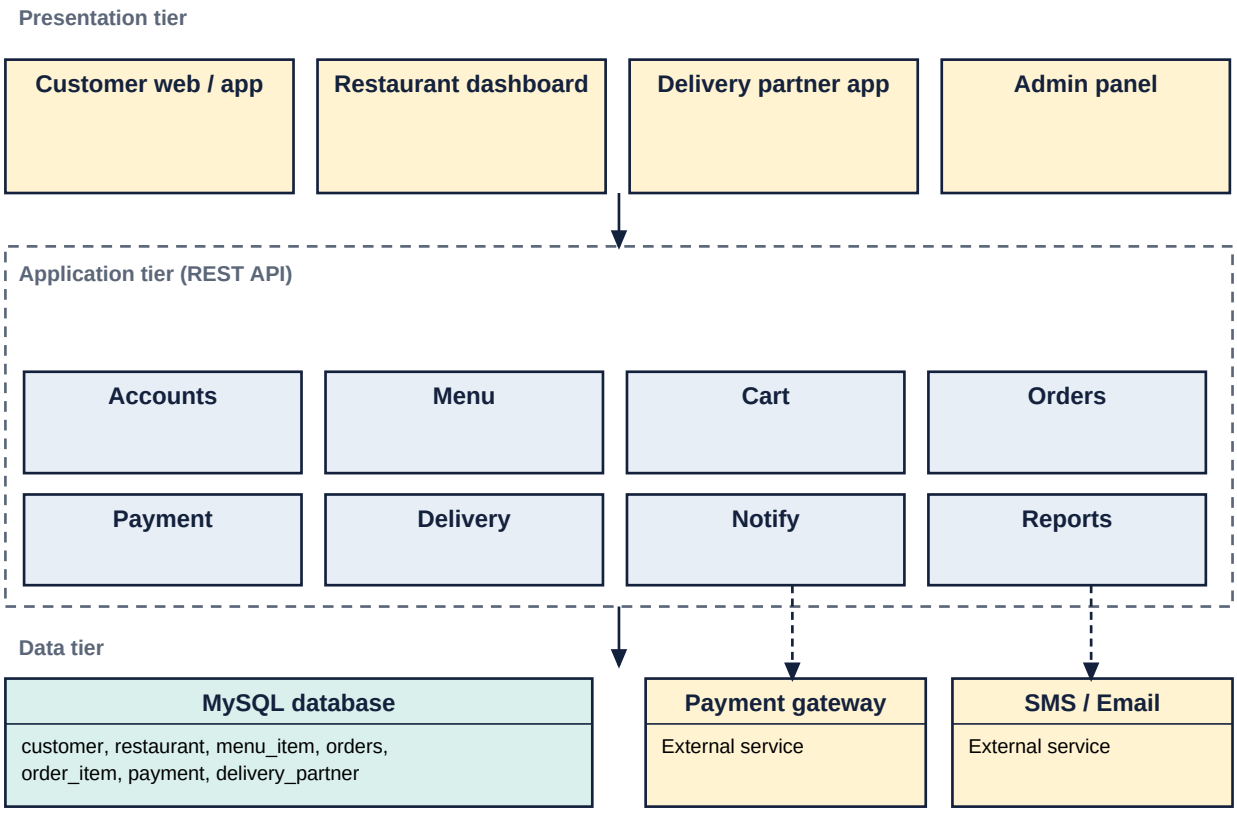

Figure 7.1: Three-tier architecture

7.2 Database Design

The ER diagram is converted to the following main tables. Each table has a primary key, and the foreign keys keep the data consistent.

Table	Main columns
customer	customer_id, name, email, phone, password_hash
restaurant	restaurant_id, name, address, phone, rating
menu_item	item_id, restaurant_id, name, price, available

Table	Main columns
orders	order_id, customer_id, restaurant_id, partner_id, total, status, created_at
order_item	order_id, item_id, quantity, price
payment	payment_id, order_id, mode, amount, status
delivery_partner	partner_id, name, phone, vehicle, status

The design is in third normal form. Prices are copied into `order_item` so that a later price change does not alter an old bill.

7.3 Order Life Cycle

An order moves through a fixed set of states, and the system allows only the changes shown in the table.

Status	Set by	Next status
Placed	Customer	Accepted or Rejected
Accepted	Restaurant	Preparing
Preparing	Restaurant	Ready for pickup
Ready for pickup	Restaurant	Out for delivery
Out for delivery	Delivery partner	Delivered
Delivered	Delivery partner	Closed
Rejected or Cancelled	Restaurant or customer	Closed

7.4 Bill Calculation

The bill is the sum of the price of each item times its quantity, plus tax and the delivery charge, minus any discount.

```
subtotal = 0
for each item in cart:
    subtotal = subtotal + item.price * item.quantity
tax = subtotal * tax_rate
if subtotal >= free_delivery_limit:
    delivery = 0
else:
    delivery = delivery_charge
total = subtotal + tax + delivery - discount
```

7.5 Interface Design

The main screens are the login page, restaurant list, menu page, cart, checkout, order tracking page, order history, the restaurant dashboard, the delivery task list and the administrator panel. The design follows a few simple rules: the main action on each screen is clear, the cart is always one tap away, text is large enough to read on a phone, and errors say what went wrong and how to fix it.

7.6 Sequence of Placing an Order

The following steps show how the parts of the system work together when a customer places an order.

Step	Actor	Action
1	Customer	Opens a restaurant, adds items to the cart and presses Checkout
2	System	Calculates the bill and shows it with the delivery address
3	Customer	Selects a payment mode and confirms
4	System	Sends the amount to the payment gateway and waits for the result
5	Payment gateway	Returns success or failure
6	System	On success, saves the order with status Placed and notifies the restaurant
7	Restaurant	Accepts the order and starts preparing it
8	System	Assigns a free delivery partner and notifies the customer
9	Delivery partner	Picks up and delivers the order, then marks it delivered

7.7 Data Dictionary

Field	Type	Meaning
customer_id, order_id, item_id	Integer	Unique numbers that identify a row
price, total, amount	Decimal	Money values with two decimal places
status (orders)	Text	One of the states in the order life cycle
mode (payment)	Text	Card, UPI, wallet or cash on delivery
available	Boolean	Whether a menu item can be ordered now
created_at	Date and time	When the order was placed

8. Technology Stack and Implementation Plan

Layer	Technology
Front end	HTML, CSS and JavaScript
Back end	A server-side language such as Python, Java or PHP with a REST API
Database	MySQL
Payment	Payment gateway in test mode during development
Notifications	SMS or email service
Version control	Git and GitHub
Testing tools	Browser developer tools and an API testing tool

The code is organised by module so that each team member can work on one module without disturbing the others. Every change goes through version control, and each sprint ends with a working version that is shown to the product owner.

9. Testing

9.1 Test Strategy

Testing is done at four levels. Unit tests check single functions, such as the bill calculation. Integration tests check that modules work together, for example that a placed order appears on the restaurant screen. System tests check the complete flow from login to delivery. User acceptance tests are carried out by real customers and restaurant staff before release.

9.2 Test Cases

ID	Test	Input	Expected result
T1	Register	Valid name, email, phone, password	Account created
T2	Register with used email	Existing email	Error message shown
T3	Login	Correct email and password	Home page opens
T4	Wrong password	Incorrect password	Login refused
T5	Search	Cuisine name	Matching restaurants listed
T6	Add to cart	Select an item	Cart count and subtotal update
T7	Bill	Three items with tax and delivery	Total matches the formula
T8	Place order	Confirm the cart	Order created with status Placed
T9	Online payment	Test card	Payment recorded as successful
T10	Failed payment	Declined test card	Order not confirmed, user told why
T11	Restaurant accepts	Press Accept	Customer sees Accepted
T12	Delivery complete	Mark delivered	Order status becomes Delivered
T13	Unauthorised access	Customer opens admin page	Access denied
T14	Empty cart	Press Checkout with no items	Message asks to add items

9.3 Defect Handling

Each defect is recorded with its steps, its severity and the person who will fix it. Serious defects such as payment errors are fixed first. After a fix, the test that found the defect is repeated, and the related tests are run again to make sure nothing else was broken.

10. Risk Analysis and Project Schedule

10.1 Risk Analysis

Risk	Likelihood	Impact	Plan
Payment gateway is unavailable	Medium	High	Offer cash on delivery and retry the payment
Heavy load at meal times	High	High	Load testing and scalable hosting
Wrong or lost orders	Low	High	Confirmation screens and order logs
Personal data is leaked	Low	High	Hashed passwords, HTTPS and access control
Requirements keep changing	High	Medium	Agile sprints and a managed backlog
Team member is unavailable	Medium	Medium	Shared code and documented modules

10.2 Project Schedule

Weeks	Activity
1–2	Requirement gathering and feasibility study
3–4	Modelling and database design
5–6	Sprint 1 and Sprint 2: accounts, menu, cart, orders
7–8	Sprint 3: payment and tracking
9–10	Sprint 4 and Sprint 5: restaurant, delivery, admin
11	System and acceptance testing
12	Fixes, documentation and release

11. Maintenance and Security

11.1 Maintenance

After release, four kinds of maintenance are needed. Corrective maintenance fixes defects found by users. Adaptive maintenance updates the system when the environment changes, for example when the payment gateway changes its interface. Perfective maintenance adds improvements that users ask for. Preventive maintenance cleans the code and updates libraries to avoid future problems. A regular backup of the database is kept so that orders can be recovered after a failure.

11.2 Security Measures

- Passwords are stored only as salted hashes.
- All pages are served over HTTPS.
- Each user role can open only its own pages.
- Input is checked and cleaned to prevent injection attacks.
- Card details are handled only by the payment gateway.
- Sessions end after a period of inactivity.

12. Learning Outcomes and Future Enhancements

12.1 Learning Outcomes

- I learned how a vague idea becomes a clear set of requirements by identifying stakeholders and their needs.
- I learned to choose a process model by comparing it with the nature of the project.
- I practised use case, data flow and entity-relationship diagrams.
- I understood why testing, risk analysis and security must be planned from the start.
- I saw that documentation and scheduling are as important as writing code.

12.2 Future Enhancements

- A mobile application for each type of user.
- Live map tracking of the delivery partner.
- Personal recommendations based on past orders.
- Coupons, offers and a loyalty programme.
- Scheduled orders and group orders.
- Support for several languages.

13. Conclusion

This case study used the methods of software engineering to plan an Online Food Ordering System. The problem was stated, the feasibility was checked, and Agile was chosen as the process model. The requirements were written down for each kind of user and modelled with use case, data flow and entity-relationship diagrams. A three-tier architecture and a relational database were designed, and plans for testing, risk handling, scheduling and maintenance were prepared.

The case study shows that a good system is the result of careful analysis and design as well as coding. Following the life cycle step by step reduces mistakes, makes the work easier to share in a team, and produces software that users can trust. The design in this report can be used as a base for building the working system in later sprints.

14. References

1. Pressman, R. S. and Maxim, B. R. Software Engineering: A Practitioner’s Approach. McGraw-Hill Education.
2. Sommerville, I. Software Engineering. Pearson.
3. Jalote, P. An Integrated Approach to Software Engineering. Springer.
4. Schwaber, K. and Sutherland, J. The Scrum Guide. scrumguides.org.
5. Elmasri, R. and Navathe, S. B. Fundamentals of Database Systems. Pearson.
6. Mozilla Developer Network. Web technology documentation. developer.mozilla.org.